

Gemini 3 Flash Preview: Optimized Schema & Implementation Guide

This guide consolidates recommendations for extracting high-fidelity structured data from documents using Google Gemini 3 Flash Preview. It addresses limitations in flat schemas and provides a production-ready, polymorphic approach.

1. The Core Philosophy: Polymorphism & Recursion

Older LLMs struggled with complex, nested JSON. Gemini 3 Flash excels at it. Instead of a flat list of generic "blocks," we use **Polymorphism**.

- **Old Way (Flat):** Every block has every field (`text` , `url` , `rows` , `level`), but most are null.
- **New Way (Polymorphic):** A block is strictly typed. A `Table` block *only* has table fields. A `Heading` block *only* has heading fields.

Benefits:

1. **Strict Validation:** The model cannot hallucinate a "level 7" heading or a table without rows.
2. **Recursion:** Lists can contain lists (perfect for nested HTML `<u1>`).
3. **Token Efficiency:** No wasted "null" fields in the output.

2. Critical Fix: The Table Schema

The Problem in Many Scripts

In many initial implementations, Tables are defined as lists of strings:

```
# ❌ INCORRECT: Cannot capture rowspans/colspans
rows: list[list[str]]
```

If a table has merged cells, the model has nowhere to put that data. It ignores your prompt instructions to "set row_span" because the schema doesn't allow it.

The Solution

Rows must be lists of **Objects**:

```
# ✅ CORRECT: Captures metadata per cell
rows: list[list[TableCell]]
```

3. Optimized Pydantic Models (Python SDK)

Replace your existing `BaseModel` definitions with this polymorphic structure. This structure uses `typing.Union` to force the model to choose a specific block type.

```

from typing import Union, List, Literal, Optional
from pydantic import BaseModel, Field

# --- Sub-components ---

class Span(BaseModel):
    """Start/end indices for styling within a text block."""
    start: int
    end: int
    style: Literal["bold", "italic", "code", "link"]
    url: Optional[str] = None

class TableCell(BaseModel):
    """A single cell in a table, supporting HTML rowspan/colspan."""
    content: str
    row_span: int = Field(default=1, description="1 for standard, >1 for merged r")
    col_span: int = Field(default=1, description="1 for standard, >1 for merged c")
    is_header: bool = Field(default=False)

class ListItem(BaseModel):
    """An item in a list, which can contain content AND a nested list."""
    content: str
    # RECURSION: A list item can contain another list
    sub_list: Optional["ListBlock"] = None

# --- Main Block Types ---

class TableBlock(BaseModel):
    type: Literal["table"] = "table"
    caption: Optional[str] = None
    headers: List[str] = Field(default_factory=list)
    rows: List[List[TableCell]] # List of Lists of Objects
    bbox_top: Optional[float] = None
    bbox_left: Optional[float] = None

class HeadingBlock(BaseModel):
    type: Literal["heading"] = "heading"
    level: int = Field(ge=1, le=6, description="HTML H1-H6")
    content: str
    id: Optional[str] = Field(description="Slug for anchor links")
    bbox_top: Optional[float] = None

class ParagraphBlock(BaseModel):
    type: Literal["paragraph"] = "paragraph"
    content: str
    spans: List[Span] = Field(default_factory=list)
    text_direction: Literal["ltr", "rtl", "auto"] = "auto"
    bbox_top: Optional[float] = None

class ListBlock(BaseModel):
    type: Literal["list"] = "list"
    style: Literal["unordered", "ordered"]
    items: List[ListItem]
    bbox_top: Optional[float] = None

class EquationBlock(BaseModel):

```

```

    type: Literal["equation"] = "equation"
    latex: str
    is_display: bool = Field(default=True)
    number: Optional[str] = None
    bbox_top: Optional[float] = None

class ImageBlock(BaseModel):
    type: Literal["image"] = "image"
    image_type: Literal["chart", "graph", "diagram", "figure", "photo"]
    description: str
    caption: Optional[str] = None
    bbox_top: float
    bbox_left: float
    bbox_width: float
    bbox_height: float
    image_data: Optional[str] = None

# --- The Union Type ---

# This forces the model to pick exactly ONE of these types per block
ContentBlock = Union[HeadingBlock, ParagraphBlock, TableBlock, ListBlock, Equatic

class PageContent(BaseModel):
    page_number: int
    header: Optional[str] = None
    footer: Optional[str] = None
    blocks: List[ContentBlock] # The main polymorphic list
    has_multi_column: bool = False

# Resolve the forward reference for recursion (ListBlock -> ListItem -> ListBlock
ListBlock.model_rebuild()
ListItem.model_rebuild()

```

4. Raw JSON Schema (For REST API / Non-Pydantic)

If you are using the raw REST API or another language, this is the JSON structure equivalent to the Pydantic models above.

```

{
  "type": "object",
  "properties": {
    "content": {
      "type": "array",
      "items": {
        "anyOf": [
          { "$ref": "#/definitions/HeadingBlock" },
          { "$ref": "#/definitions/ParagraphBlock" },
          { "$ref": "#/definitions/ListBlock" },
          { "$ref": "#/definitions/TableBlock" },
          { "$ref": "#/definitions/EquationBlock" }
        ]
      }
    }
  },
  "definitions": {
    "HeadingBlock": {

```

```

    "type": "object",
    "properties": {
      "type": { "const": "heading" },
      "level": { "type": "integer" },
      "text": { "type": "string" }
    },
    "required": ["type", "level", "text"]
  },
  "TableBlock": {
    "type": "object",
    "properties": {
      "type": { "const": "table" },
      "rows": {
        "type": "array",
        "items": {
          "type": "array",
          "items": {
            "type": "object",
            "properties": {
              "content": { "type": "string" },
              "row_span": { "type": "integer" },
              "col_span": { "type": "integer" }
            }
          }
        }
      }
    },
    "required": ["type", "rows"]
  },
  "ListBlock": {
    "type": "object",
    "properties": {
      "type": { "const": "list" },
      "style": { "enum": ["unordered", "ordered"] },
      "items": {
        "type": "array",
        "items": {
          "type": "object",
          "properties": {
            "content": { "type": "string" },
            "sub_list": { "$ref": "#/definitions/ListBlock" }
          }
        },
        "required": ["content"]
      }
    },
    "required": ["type", "style", "items"]
  }
}

```

5. Updated System Prompt

With polymorphic schemas, you should update your system instructions to explicitly reference the types.

Prompt Snippet:

"You are a document extraction engine. Output a list of strictly typed content blocks:

1. **Headings:** Use the 'heading' type. Determine level (1-6) based on font size and hierarchy.
2. **Paragraphs:** Use 'paragraph' type for standard text.
3. **Tables:** Use 'table' type. **CRITICAL:** You must populate the 'rows' with 'TableCell' objects. If a cell spans multiple rows or columns in the PDF, you must set `row_span` or `col_span > 1`.
4. **Lists:** Use 'list' type. If a list item has a nested sub-list, create a new 'ListBlock' object inside the 'sub_list' field of that item. Do not just flatten the list."

6. HTML Renderer Implementation

Your `_render_page` function becomes a cleaner dispatcher that switches on object type.

```
def _render_block(self, block: ContentBlock) -> str:
    """Recursive render function."""

    # 1. Heading
    if isinstance(block, HeadingBlock):
        return f'<h{block.level} id="{block.id or ""}">{self._escape(block.content)}</h{block.level}>'

    # 2. Paragraph
    elif isinstance(block, ParagraphBlock):
        # Handle direction
        dir_attr = f' dir="{block.text_direction}"' if block.text_direction != "c" else ""
        return f'<p{dir_attr}>{self._escape(block.content)}</p>'

    # 3. Table (With Merge Support)
    elif isinstance(block, TableBlock):
        html = ["<table>"]
        if block.caption:
            html.append(f"<caption>{self._escape(block.caption)}</caption>")

        # Headers
        if block.headers:
            html.append("<thead><tr>")
            for h in block.headers:
                html.append(f"<th>{self._escape(h)}</th>")
            html.append("</tr></thead>")

        # Rows
        html.append("<tbody>")
        for row in block.rows:
            html.append("<tr>")
            for cell in row:
                attrs = []
                if cell.row_span > 1: attrs.append(f'rowspan="{cell.row_span}"')
                if cell.col_span > 1: attrs.append(f'colspan="{cell.col_span}"')
                tag = "th" if cell.is_header else "td"
                html.append(f'<{tag} {" ".join(attrs)}>{self._escape(cell.content)}</{tag}>')
            html.append("</tr>")
        html.append("</tbody></table>")
        return "".join(html)
```

```
# 4. List (Recursive)
elif isinstance(block, ListBlock):
    tag = "ul" if block.style == "unordered" else "ol"
    html = [f"<{tag}>"]
    for item in block.items:
        html.append(f"<li>{self._escape(item.content)}")
        # Check for recursion
        if item.sub_list:
            html.append(self._render_block(item.sub_list))
        html.append("</li>")
    html.append(f"</{tag}>")
    return "".join(html)

# 5. Equation
elif isinstance(block, EquationBlock):
    tag = "div" if block.is_display else "span"
    cls = "equation-display" if block.is_display else "equation-inline"
    # Wrap LaTeX
    wrapper = f"\\[{{block.latex}}\\]" if block.is_display else f"\\({{block.latex}}\\)"
    return f'<{tag} class="{cls}">{wrapper}</{tag}>'

return ""
```